A Retrieval-Augmented Language Model Assistant for Linux Manual Pages

Frederico Pinto (129466)
Marcel Schwemin (130280)
Department of Electronics Telecommunications and Informatics
University of Aveiro
Class: *Foundations of Machine Learning*
Instructor: Petia Georgieva
Email: fredericopinto@ua.pt, marcel17@ua.pt

***Abstract*—Linux manual pages (*man pages*) provide comprehensive documentation for Unix-based systems, yet their usage remains limited by poor discoverability and steep learning curves, particularly for novice users. This project explores the development of a Retrieval-Augmented Generation (RAG) based assistant that enables natural language querying over the full corpus of Linux man pages. The system combines structured data extraction, semantic indexing, and an open-source large language model to provide accurate, context-aware responses without retraining the base model. This report presents the motivation, data processing pipeline, and machine learning formulation of the problem, and outlines the applied methods and evaluation strategy.**

***Index Terms*—Retrieval-Augmented Generation, Large Language Models, Linux Documentation, Information Retrieval, Natural Language Processing**

—

I. Introduction

Linux manual pages are the primary source of authoritative documentation for Unix commands. While exhaustive, they are designed for linear reading and keyword-based lookup, which makes them inefficient for exploratory or task-oriented queries. Modern language models have demonstrated strong capabilities in natural language understanding and question answering; however, directly training or fine-tuning large language models on extensive collections of structured technical documentation, such as Linux manual pages, is computationally expensive and inflexible.

This project addresses these challenges by constructing a domain-specific RAG assistant over the complete local man-page database.

—

II. State of the Art

Recent advances in natural language processing have been driven by large language models (LLMs), which demonstrate strong capabilities in question answering, summarization, and dialogue across a wide range of domains. Despite this progress, purely generative LLMs frequently produce plausible but incorrect statements ("hallucinations"), particularly on narrow technical subjects that require precise, verifiable facts.

A. Retrieval-Augmented Generation and grounding

Retrieval-Augmented Generation (RAG) was introduced to address the problem of grounding generative models in external knowledge sources. Lewis et al. propose a RAG architecture that retrieves relevant passages and conditions generation on those passages to improve factual accuracy and updateability [1]. Building on this idea, Izacard and Grave proposed the Fusion-in-Decoder (FiD) architecture, showing that jointly conditioning on multiple retrieved passages in the decoder can further improve answer quality in open-domain QA tasks [2].

B. Dense retrieval and semantic indexing

Dense retrieval methods embed queries and documents into a shared vector space and perform nearest-neighbor search for retrieval. Karpukhin et al. introduced Dense Passage Retrieval (DPR), which demonstrated strong retrieval performance for open-domain QA using learned dense encoders [3]. For sentence- and passage-level semantic embeddings, Sentence-BERT (SBERT) provides efficient and effective sentence embeddings that are widely used in retrieval pipelines [4]. Practical, large-scale similarity search is commonly implemented with FAISS [6] to enable efficient nearest-neighbor retrieval over millions of vectors.

C. Hallucination mitigation via retrieval

While retrieval augmentation reduces the tendency of LLMs to hallucinate, prompt-only constraints are often insufficient. Shuster et al. provide empirical evidence that retrieval augmentation reduces hallucinations in conversational systems, but also highlight that additional mechanisms—such as stricter grounding checks or refusal policies—are needed for safety-critical domains [5]. These findings motivate design choices that combine dense retrieval with explicit retrieval-confidence gating and refusal behavior.

D. Technical documentation and domain-specific assistants

Traditional documentation access tools (e.g., `man` and `apropos`) rely on keyword lookup and linear reading. Recent applied work on documentation and code assistants shows the value of combining retrieval with generation for developer

productivity, but such systems often trade off coverage for grounding guarantees. The literature above (RAG, FiD, DPR, SBERT, and hallucination studies) provides the methodological foundation for a documentation-focused RAG assistant that emphasizes correctness and verifiability.

—

III. Problem Complexity and Motivation

The primary challenge of this problem lies in the complexity of the data rather than model architecture. Linux man pages are written in *roff* markup and rendered into terminal-formatted text, resulting in wrapped lines, indentation-based structure, and mixed semantic content. Extracting meaningful, retrieval-ready chunks requires non-trivial preprocessing and structural inference.

From a machine learning perspective, the problem is not traditional supervised learning, but rather a combination of information retrieval, semantic similarity search, and conditional text generation. The difficulty arises from ensuring that retrieved contexts are both relevant and sufficiently precise to avoid hallucinations by the language model.

—

IV. Data Description and ML Problem Formulation

A. Data Source

The dataset consists of the complete set of Linux manual pages available on a local system, indexed via the `mandb` database. Approximately 21,000 individual man pages were identified and extracted, covering commands, system calls, configuration files, and library functions.

B. Machine Learning Problem

The task is formulated as a **retrieval-augmented question answering** problem.

- **Input**: A natural language user query.
- **Intermediate Input**: Retrieved text chunks from the man-page corpus.
- **Output**: A natural language answer grounded in retrieved documentation.

This is neither a pure classification nor regression task, but a hybrid NLP system combining dense retrieval and conditional text generation.

—

V. Data Preprocessing

This section describes the complete data acquisition and preprocessing pipeline used to transform raw Linux manual pages into a structured corpus suitable for retrieval-augmented language modeling.

A. Data Gathering and Corpus Construction

The data source for this project consists of the complete set of Linux manual pages available on the target system. Rather than relying on a predefined list of commands, the corpus was constructed programmatically using the system-wide manual page index generated by `mandb`.

The `apropos` command, which queries the `mandb` database, was used to enumerate all available manual page entries. This approach ensures comprehensive coverage of installed documentation, including user commands, system calls, library functions, configuration files, and administrative utilities. By querying the index rather than manually scanning directories, the data collection process remains robust to variations in system configuration and installed packages.

Each unique manual page identifier extracted via `apropos` was treated as a separate document. In total, approximately 21,000 man pages were identified and processed in the development computer, forming a large-scale technical text corpus.

B. Man Page Extraction

Manual pages are authored in *roff* markup and rendered dynamically when viewed. To obtain a stable and parseable textual representation, each man page was rendered using the `man` command with a fixed output width:

```
MANWIDTH=1000 man -P cat <page>
```

Setting a large fixed width minimizes line wrapping artifacts that would otherwise interfere with structural parsing, particularly for option descriptions and inline flag references. Output was captured directly from standard output and formatting artifacts such as backspaces and overstrikes were removed.

C. Formatting Normalization

The rendered man-page output contains hard line breaks introduced for terminal display purposes. These line breaks do not correspond to semantic boundaries and therefore require normalization.

Text was grouped into paragraphs based on:

- Blank line separation
- Consistent indentation level across consecutive lines

Wrapped lines belonging to the same paragraph were merged into a single continuous text segment. This reflowing step significantly improves semantic coherence and embedding quality by restoring complete sentences and logical units of meaning.

D. Structural Parsing and Option Identification

Linux man pages rely heavily on indentation to convey document structure. This property was exploited to infer semantic roles within the text. An indentation-aware parsing strategy was employed to distinguish between:

- Option headers (e.g., `-L`, `--help`)
- Option-specific descriptions
- Section-level contextual explanations

This approach prevents the erroneous association of global behavioral descriptions or cross-option interactions with individual flags.

E. Chunking Strategy

Following structural parsing, the text was segmented into semantically meaningful chunks suitable for retrieval. Two primary chunk types were created:

- **Option chunks**: describing a single command-line option and its behavior
- **Context chunks**: describing general behavior, constraints, or interactions between options

Chunks were designed to balance completeness with conciseness, ensuring that each chunk contains sufficient information to answer a user query while remaining small enough for efficient embedding and retrieval.

Each chunk was stored together with metadata including:

- Command name
- Manual section
- Option identifier (when applicable)

F. Dataset Usage and Validation Strategy

As the system is based on retrieval-augmented generation rather than supervised learning, traditional train/validation/test splits are not directly applicable. Instead, evaluation focuses on retrieval quality and end-to-end question answering performance using held-out query sets.

Cross-validation in the classical sense is therefore replaced by repeated evaluation across diverse query categories, including option-specific questions, behavioral explanations, and comparative queries involving multiple flags.

—

VI. Applied Machine Learning Methods

This section describes the machine learning components used in the proposed Retrieval-Augmented Generation (RAG) system, including text embedding, vector indexing, retrieval, and language model inference. The system is designed to answer natural language queries over Linux manual pages without retraining or fine-tuning the underlying language model.

A. System Overview

The proposed assistant follows a standard RAG pipeline. Given a user query, the system retrieves the most semantically relevant documentation fragments from a vector index built over the man-page corpus. These retrieved fragments are then provided as context to a large language model, which generates a final response grounded in the retrieved documentation.

This approach separates knowledge storage from language generation, enabling efficient updates to the knowledge base and reducing computational requirements compared to full model fine-tuning.

B. Text Embedding Model

To enable semantic retrieval, all documentation chunks are embedded into a fixed-dimensional vector space using the `SentenceTransformers` library. Specifically, the pre-trained `all-MiniLM-L6-v2` model was employed.

This model maps text segments into 384-dimensional dense vectors and is optimized for semantic similarity tasks. It offers a favorable trade-off between computational efficiency and embedding quality, making it well suited for large-scale technical corpora such as Linux manual pages.

All embeddings are L2-normalized prior to indexing, allowing cosine similarity to be computed efficiently via inner product.

C. Document Chunking

Each parsed man page is flattened into plain text and divided into overlapping chunks of fixed length. Chunking is performed using a sliding window approach with a chunk size of 500 characters and an overlap of 100 characters.

The overlap ensures that important information spanning chunk boundaries is preserved, reducing the risk of semantic fragmentation. This strategy balances retrieval granularity with embedding efficiency and is commonly employed in retrieval-based NLP systems.

D. Vector Indexing and Similarity Search

The embedded document chunks are stored in a FAISS vector index (`IndexFlatIP`), which performs exact nearest-neighbor search using inner product similarity. Given that embeddings are normalized, this corresponds to cosine similarity.

FAISS was selected due to its efficiency, scalability, and widespread adoption in large-scale information retrieval applications. The index enables fast retrieval of the top-k most relevant chunks for a given query embedding.

Index persistence is supported by serializing both the FAISS index and associated metadata, allowing the system to be reloaded without recomputing embeddings.

E. Query Processing and Retrieval

When a user submits a query, the query text is embedded using the same sentence transformer model used during indexing. The resulting embedding is used to perform a similarity search against the FAISS index.

The top-k most similar document chunks are retrieved and concatenated to form a contextual prompt. In this implementation, $k = 3$ was selected empirically to balance contextual coverage with prompt length constraints.

F. Language Model Inference

The generation component of the system is handled by an open-source large language model accessed via the `Ollama` framework. The retrieved documentation chunks are injected into a structured prompt that explicitly instructs the model to answer the query using only the provided context.

This constraint is intended to reduce hallucinations and ensure that generated responses remain grounded in authoritative documentation. The language model is not fine-tuned on the man-page corpus; instead, it operates purely in an inference-only mode.

G. Retrieval-Augmented Generation Strategy

The overall system implements a classical RAG architecture, where retrieval and generation are decoupled. This design offers several advantages:

- New documentation can be incorporated without retraining the model.
- Errors in generation can be traced back to retrieval quality.
- The system remains computationally lightweight and deployable on commodity hardware.

By combining dense semantic retrieval with controlled prompt-based generation, the assistant is able to provide accurate and context-aware responses to technical queries about Linux commands.

H. Language Model Inference

The generation component of the system is handled by an open-source large language model accessed via the `Ollama` framework. Several candidate models were tested during development. Among these, `llama3.1` was selected for the final system configuration due to its more consistent response quality when answering documentation-related queries, although other models can be selected using the OLLAMA_MODEL environment variable.

Empirically, `llama3.1` produced fewer unsupported inferences and adhered more reliably to prompt constraints when compared to smaller or older models, while maintaining interactive response times. On the evaluation system, responses were typically generated within a few seconds, which was deemed acceptable for an interactive documentation assistant.

All experiments were conducted on a local workstation equipped with an NVIDIA RTX 3060 Ti GPU and 16 GB of system memory. This hardware configuration allowed the system to operate fully offline without requiring remote inference services, demonstrating the practicality of the proposed approach on commodity consumer hardware.

The language model operates purely in inference mode and is not fine-tuned on the man-page corpus. Instead, grounding and correctness are enforced through retrieval-augmented prompting and retrieval-confidence gating, as described in the following sections.

—

VII. Results and Evaluation

The proposed system was evaluated qualitatively through interactive question-answering sessions covering common Linux usage scenarios. Given the absence of labeled ground-truth answers for the man-page corpus, evaluation focused on response correctness, grounding in retrieved documentation, and practical usefulness to end users.

A. Retrieval Gating and Hallucination Suppression

A key design objective of the proposed system is to ensure that responses are generated only when supported by retrieved documentation. To evaluate this behavior, the system was tested with out-of-domain queries that cannot be answered using Linux manual pages.

1) Behavior Without Retrieval Gating: Before enforcing a retrieval-confidence threshold, the system was prone to generating fluent but unsupported responses for queries unrelated to system documentation. An example interaction is shown below.

Listing 1. Example of hallucinated response without retrieval gating

```
You: What is the capital of the USA?

Bot: To find the capital of the USA, I can use
     a hash to store country names as keys and
     their corresponding cities as values.
    Then, I can search for the list associated
     with the USA key.

Here's an example:

#!/bin/bash

declare -A countries
countries[Finland]="Helsinki"
countries["Germany"]="Berlin Frankfurt"
countries["USA"]="Chicago New York Washington"

echo "${countries['USA']}" | tr ' ' '\n'

ref: tr(1)
```

Although syntactically plausible, the response is not grounded in any relevant Linux manual page and incorrectly references `tr(1)`. This constitutes a hallucinated answer produced in the absence of relevant retrieved documentation.

2) Behavior With Retrieval Gating Enabled: After introducing a similarity-based retrieval gate, the system behavior changed significantly. For the same out-of-domain query, the assistant correctly detected the absence of relevant documentation and declined to answer, as shown below.

Listing 2. Correct refusal with retrieval gating enabled

```
You: What is the capital of the USA?

Bot: The information about country capitals is
     not available in this context.

(man: none relevant)
```

Rather than producing an unsupported response, the system explicitly indicates that the requested information falls outside the scope of the man-page corpus. This behavior demonstrates effective hallucination suppression and ensures that generated answers remain strictly grounded in retrieved documentation.

3) Discussion: This comparison highlights the importance of enforcing retrieval-based constraints in Retrieval-Augmented Generation systems. By preventing the language model from responding when retrieval confidence is low, the proposed approach maintains a strict coupling between answers and authoritative documentation. Such behavior is particularly desirable in technical assistance settings, where incorrect or fabricated information may lead to user confusion or system misuse.

B. Qualitative Evaluation

Given the absence of labeled ground-truth question–answer pairs for the Linux man-page corpus, system performance was evaluated qualitatively through interactive testing. The evaluation focused on response correctness, grounding in retrieved documentation, and practical usefulness to end users.

Queries were manually designed to reflect realistic user scenarios, including command usage, option discovery, and system administration tasks. Example interactions are presented below to illustrate typical system behavior.

1) In-Domain Query Example: Listing 3 shows an example of an in-domain query related to mounting removable storage devices. The system retrieves relevant documentation and produces a response grounded in Linux manual pages.

Listing 3. Qualitative evaluation: in-domain query

```
You: How do I mount a USB drive?

Bot: To mount a USB drive, you can use the
    udisksctl command with the
mount option.

First, identify the block device or object
    path of your USB drive using:

$ udisksctl info --block-device /dev/sdb1

Then mount the device using:

$ udisksctl mount --object-path \
/org/freedesktop/UDisks2/block_devices/sdb1

Alternatively, the mount command can be used:

$ sudo mount /dev/sdb1 /media/myusbdrive

References:
- udisksctl(1)
- mount(8)
```

The response correctly identifies relevant commands, provides actionable instructions, and explicitly references the corresponding manual pages. This demonstrates the system's ability to synthesize information from multiple documentation sources while remaining grounded in retrieved context.

2) Out-of-Domain Query Example: To further assess robustness, the system was evaluated using queries outside the scope of Linux manual pages. Listing 4 presents an example of such an interaction.

Listing 4. Qualitative evaluation: out-of-domain query

```
You: What is the capital of the USA?

Bot: The information about country capitals is
     not available in this
context.

(man: none relevant)
```

In this case, the system correctly refrains from generating an unsupported answer. This behavior confirms that retrieval gating effectively limits responses to information contained within the man-page corpus, thereby preventing hallucinations.

3) Discussion: These qualitative results indicate that the proposed system performs reliably for documentation-related queries while safely declining to answer questions beyond its knowledge scope. The evaluation demonstrates that retrieval-augmented generation, combined with retrieval-confidence gating, produces responses that are both useful and trustworthy in a technical assistance setting.

C. Scalability and Performance

The system successfully indexed approximately 605,000 documentation chunks derived from roughly 21,000 Linux manual pages. Index construction is performed offline and persisted, enabling fast startup times for subsequent sessions.

At inference time, query embedding, retrieval, and response generation occur within interactive latency bounds on commodity hardware, making the approach suitable for local deployment without specialized accelerators.

D. Limitations

Despite its effectiveness, the system exhibits several limitations. Retrieval quality depends on the embedding model's ability to capture technical semantics, and occasionally relevant context may be omitted from the top-k results. Additionally, prompt length constraints limit the amount of documentation that can be supplied to the language model, which may affect responses to highly complex queries.

These limitations are inherent to retrieval-based systems and suggest directions for future improvements, such as adaptive chunk selection or re-ranking strategies.

—

VIII. Novelty and Contributions

This project makes several contributions in the context of retrieval-augmented language model systems applied to technical documentation. While Retrieval-Augmented Generation (RAG) itself is well established in the literature, the novelty of this work lies in the careful adaptation and constraint of the approach to a highly structured, safety-critical documentation domain.

A. Structured Parsing of Man Pages

A key contribution of this work is the development of a preprocessing pipeline that leverages indentation-aware parsing to preserve the semantic structure of Linux manual pages. Unlike naive approaches that treat documentation as unstructured text, the proposed method distinguishes between option-specific descriptions and section-level contextual explanations. This separation improves retrieval precision and prevents the erroneous association of global behavioral descriptions with individual command options.

B. Retrieval-Confidence Gating for Hallucination Prevention

A central novelty of the system is the explicit enforcement of a retrieval-confidence threshold prior to invoking the language model. Many RAG implementations rely solely on prompt-based instructions to restrict model behavior; however, such approaches do not reliably prevent hallucinated responses. In contrast, the proposed system uses vector similarity scores to determine whether sufficient relevant documentation is available. When retrieval confidence falls below a predefined threshold, the system declines to answer.

This design choice ensures that the assistant operates strictly within the bounds of its documentation corpus and avoids generating unsupported or misleading responses. The effectiveness of this mechanism is demonstrated through qualitative examples comparing system behavior before and after retrieval gating.

C. Transparent and Verifiable Responses

Another contribution of this work is the emphasis on transparency through explicit manual page references in generated responses. By requiring the language model to cite the relevant man pages for each command mentioned, the system enables users to verify answers against authoritative sources. This feature differentiates the assistant from generic conversational agents and aligns it more closely with professional documentation tools.

D. Comparison with Existing Approaches

Compared to traditional tools such as `man` and `apropos`, the proposed system offers improved accessibility by supporting natural language queries and synthesizing information across multiple manual pages. Unlike general-purpose language model assistants, it avoids relying on implicit world knowledge and instead operates purely on locally available documentation.

Relative to existing RAG-based documentation assistants described in the literature, this work emphasizes stricter grounding guarantees and refusal behavior when documentation support is insufficient. While this may reduce answer coverage, it significantly improves trustworthiness, which is particularly important in technical and system administration contexts.

E. Summary of Contributions

In summary, the main contributions of this project are:

- An indentation-aware parsing strategy tailored to Linux manual pages.
- A retrieval-confidence gating mechanism that suppresses hallucinated responses.
- A transparent response format with explicit documentation references.
- A practical, lightweight RAG system deployable on commodity hardware without model fine-tuning.

These contributions demonstrate that careful system design and constraint enforcement can substantially improve the reliability of retrieval-augmented language model assistants for technical documentation.

—

IX. Conclusion

This project demonstrated the feasibility and effectiveness of a retrieval-augmented language model assistant for querying Linux manual pages using natural language. By combining structured preprocessing, dense semantic retrieval, and constrained language model generation, the system provides accurate and transparent responses grounded in authoritative documentation.

A key outcome of this work is the demonstration that strict retrieval-based constraints, including similarity threshold gating and explicit refusal behavior, are essential for preventing hallucinations in technical documentation assistants. The resulting system behaves as a reliable support tool rather than a general-purpose conversational agent, prioritizing correctness and trustworthiness over coverage.

Overall, the project highlights how careful system design and preprocessing choices can significantly improve the practical applicability of large language models in technical domains without requiring expensive model retraining.

—

References

[1] P. Lewis, E. Perez, A. Piktus, et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[2] G. Izacard and E. Grave, "Leveraging Passage Retrieval with Generative Models for Open Domain Question Answering," in *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2021.

[3] V. Karpukhin, B. Oguz, S. Min, et al., "Dense Passage Retrieval for Open-Domain Question Answering," in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2020.

[4] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2019.

[5] K. Shuster, J. Weston, E. Dinan, et al., "Retrieval Augmentation Reduces Hallucination in Conversation," in *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2021.

[6] J. Johnson, M. Douze, and H. Jégou, "Billion-Scale Similarity Search with GPUs," *IEEE Transactions on Big Data*, vol. 7, no. 3, pp. 535–547, 2019.